

CUDA Lab 4

Dries Van de Velde, Behzad Iqbal

March 2024

1 Introduction

In this lab, we learned about optimising GPU programs for speed by focusing on memory access patterns and utilising different memory types in CUDA. We explored two main aspects: improving image processing performance by optimising memory access patterns and enhancing matrix multiplication speed through the use of various memory types.

2 Coalesced memory access

We designed a kernel to calculate the average grayscale of an image, comparing coalesced and non-coalesced memory access patterns. Coalesced memory access, where data is tightly packed, was expected to be faster due to optimised memory access. However, in our results (Figure 1), we observed negligible differences between coalesced and non-coalesced memory access. This could be attributed to the data still being somewhat grouped together even in non-coalesced access, suggesting a need for more randomised data distribution to obtain significant differences.

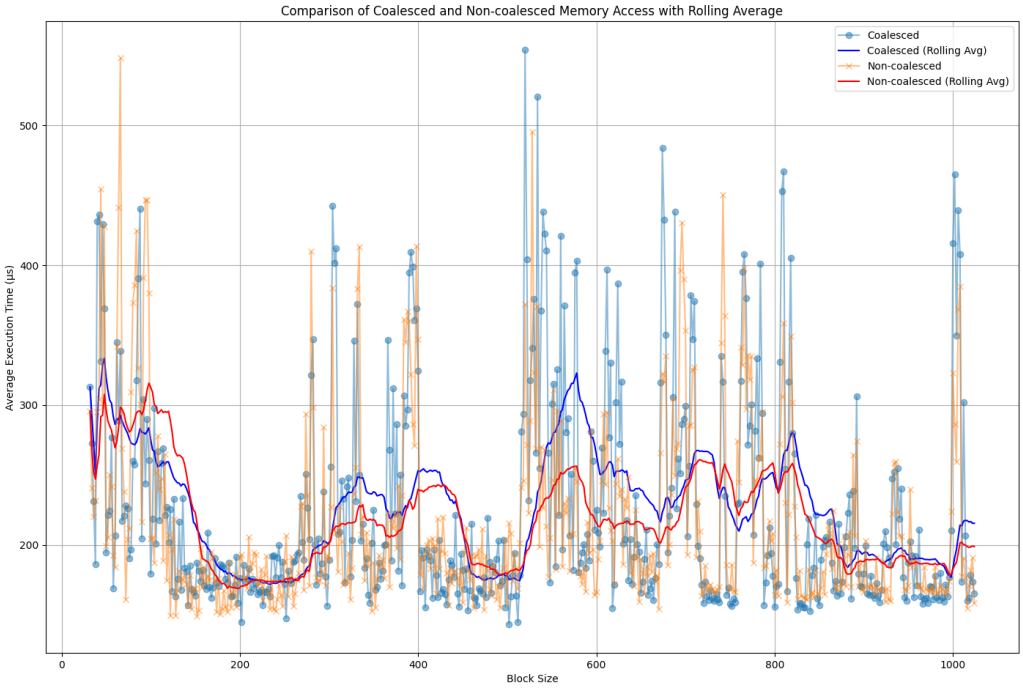


Figure 1: Block Size vs Execution Time

3 Global, Shared, and Constant Memory

In this section, we implemented a matrix multiplication algorithm using global, shared, and constant memory types. Initially, we expected shared memory to outperform global memory due to its faster access speed, while constant memory was anticipated to excel in read-heavy operations. However, our results (Figure 2) revealed that shared memory outperformed constant memory, contrary to our expectations. Furthermore, constant memory had limitations, as it wasn't plotted for matrix sizes above 512, highlighting its restricted capacity. Utilising tiling with shared memory on the CPU could potentially address this limitation, albeit at the expense of introducing performance drawbacks hence the multiple GPU calls. Interestingly, optimising memory access patterns (indicated by the '2d' suffix) had a significant impact on performance, overshadowing differences between memory types. This observation suggests that proper memory access patterns are crucial for achieving optimal GPU performance. Textured memory could be a potential solution to solve the slow constant memory implementation. Textured memory combines the advantages of both constant and global memory since it can be textured as the name implicates, offering improved performance based on our findings.

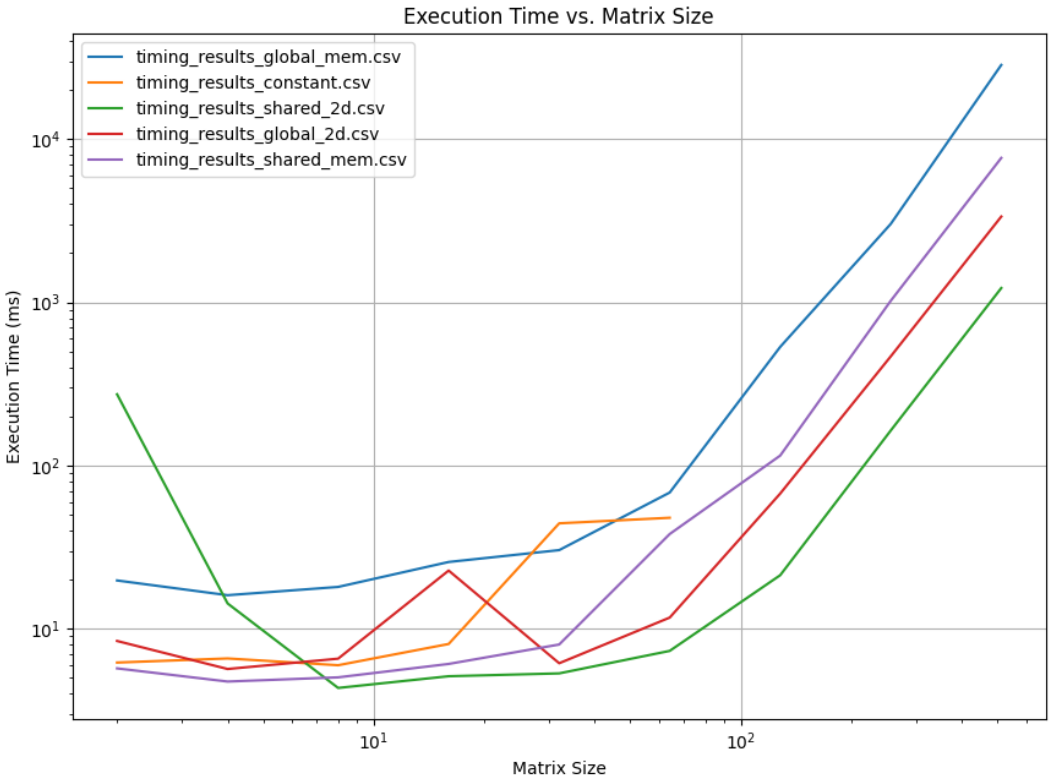


Figure 2: Execution Times vs Matrix Sizes (log scale)

4 Conclusion

In conclusion, optimising memory access patterns and selecting appropriate memory types are crucial for maximising the GPU performance. While coalesced memory access and memory type selection showed promising improvements in theory, our practical results highlighted the importance of data distribution and access patterns.